Part 4. 실습 가이드 — PyTorch 구현과 학습 팁

---

1. 환경 설정

1.1 필수 라이브러리

```
# 기본 ML
pip install torch torchvision torchaudio
pip install transformers accelerate bitsandbytes
pip install wandb tensorboard

# 로봇 학습
pip install gymnasium robosuite mujoco
pip install dm_control

# 오디오
pip install librosa soundfile audiocraft

# 유틸리티
pip install einops timm safetensors
```

1.2 GPU 요구사항

모델	추론	학습 (Full)	학습 (LoRA)
RT1 (35M)	4GB	8GB	-
OpenVLA (7B)	16GB	64GB+	24GB
π0	24GB	128GB+	32GB

---

2. VAE 완전 구현

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class AudioVAE(nn.Module):
    """오디오용 VAE - 다중해상도 STFT Loss 포함"""

    def __init__(self, in_channels=1, latent_dim=64, hidden_dims=[32
        super().__init__()
        self.latent_dim = latent_dim

        # 인코더
        encoder_layers = []
        ch = in_channels
        for h_dim in hidden_dims:
            encoder_layers.extend([
                nn.Conv1d(ch, h_dim, kernel_size=4, stride=2, paddi
                nn.BatchNorm1d(h_dim),
                nn.LeakyReLU(0.2)
            ])
            ch = h_dim
        self.encoder = nn.Sequential(*encoder_layers)

        # 잠재 공간
        self.fc_mu = nn.Linear(hidden_dims[-1], latent_dim)
        self.fc_logvar = nn.Linear(hidden_dims[-1], latent_dim)
        self.fc_decode = nn.Linear(latent_dim, hidden_dims[-1])

        # 디코더
        decoder_layers = []
        hidden_dims_rev = hidden_dims[::-1]
        for i in range(len(hidden_dims_rev) - 1):
            decoder_layers.extend([
                nn.ConvTranspose1d(hidden_dims_rev[i], hidden_dims_r
                                   kernel_size=4, stride=2, padding=
                nn.BatchNorm1d(hidden_dims_rev[i+1]),
                nn.LeakyReLU(0.2)
            ])
        decoder_layers.append(
            nn.ConvTranspose1d(hidden_dims_rev[-1], in_channels,
                               kernel_size=4, stride=2, padding=1)
        )
        self.decoder = nn.Sequential(*decoder_layers)

    def encode(self, x):
        h = self.encoder(x)
        h = h.mean(dim=-1)  # Global average pooling
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
```

```
        return mu + eps * std

    def decode(self, z):
        h = self.fc_decode(z).unsqueeze(-1)
        return torch.tanh(self.decoder(h))

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar


def vae_loss(x, x_recon, mu, logvar, beta=0.001):
    """VAE Loss = Reconstruction + β * KL"""
    # MSE reconstruction
    recon_loss = F.mse_loss(x_recon, x, reduction='sum')

    # KL divergence: -0.5 * sum(1 + log(σ²) - μ² - σ²)
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(

    return recon_loss + beta * kl_loss, recon_loss, kl_loss


class MultiResolutionSTFTLoss(nn.Module):
    """다중 해상도 STFT Loss"""

    def __init__(self, fft_sizes=[512, 1024, 2048],
                 hop_sizes=[50, 120, 240],
                 win_lengths=[240, 600, 1200]):
        super().__init__()
        self.fft_sizes = fft_sizes
        self.hop_sizes = hop_sizes
        self.win_lengths = win_lengths

    def forward(self, x, x_recon):
        total_loss = 0
        for fft, hop, win in zip(self.fft_sizes, self.hop_sizes, se
            x_stft = torch.stft(x.squeeze(1), fft, hop, win,
                                window=torch.hann_window(win, device=
                                return_complex=True)
            x_recon_stft = torch.stft(x_recon.squeeze(1), fft, hop,
                                      window=torch.hann_window(win,
                                      return_complex=True)

            x_mag = torch.abs(x_stft)
            x_recon_mag = torch.abs(x_recon_stft)

            # Spectral convergence
            sc = torch.norm(x_mag - x_recon_mag, p='fro') / (torch.

            # Log magnitude
            lm = F.l1_loss(torch.log(x_mag + 1e-7), torch.log(x_rec
```

```
            lm = F.l1_loss(torch.log(x_mag + 1e-7), torch.log(x_rec
            total_loss += sc + lm
        return total_loss / len(self.fft_sizes)
```

3. Diffusion Model 구현

```python
import math

class SinusoidalPosEmb(nn.Module):
    """타임스텝 임베딩"""
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, t):
        half_dim = self.dim // 2
        emb = math.log(10000) / (half_dim - 1)
        emb = torch.exp(torch.arange(half_dim, device=t.device) * -e
        emb = t[:, None] * emb[None, :]
        return torch.cat([torch.sin(emb), torch.cos(emb)], dim=-1)


class SimpleDiffusion(nn.Module):
    """간단한 1D Diffusion Model"""

    def __init__(self, input_dim, hidden_dim=256, T=1000):
        super().__init__()
        self.T = T

        # 노이즈 스케줄
        betas = torch.linspace(1e-4, 0.02, T)
        alphas = 1 - betas
        alphas_cumprod = torch.cumprod(alphas, dim=0)

        self.register_buffer('betas', betas)
        self.register_buffer('alphas', alphas)
        self.register_buffer('alphas_cumprod', alphas_cumprod)
        self.register_buffer('sqrt_alphas_cumprod', torch.sqrt(alpha
        self.register_buffer('sqrt_one_minus_alphas_cumprod', torch

        # 노이즈 예측 네트워크
        self.time_embed = nn.Sequential(
            SinusoidalPosEmb(hidden_dim),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim)
        )

        self.net = nn.Sequential(
            nn.Linear(input_dim + hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, input_dim)
        )
```

```python
    def forward(self, x, t):
        t_emb = self.time_embed(t)
        return self.net(torch.cat([x, t_emb], dim=-1))

    def q_sample(self, x0, t, noise=None):
        """Forward process: x0 → xt"""
        if noise is None:
            noise = torch.randn_like(x0)

        sqrt_alpha = self.sqrt_alphas_cumprod[t][:, None]
        sqrt_one_minus_alpha = self.sqrt_one_minus_alphas_cumprod[t

        return sqrt_alpha * x0 + sqrt_one_minus_alpha * noise

    def loss(self, x0):
        """학습 Loss"""
        t = torch.randint(0, self.T, (x0.shape[0],), device=x0.devi
        noise = torch.randn_like(x0)
        x_t = self.q_sample(x0, t, noise)
        noise_pred = self.forward(x_t, t.float())
        return F.mse_loss(noise_pred, noise)

    @torch.no_grad()
    def sample(self, shape, device):
        """DDPM Sampling"""
        x = torch.randn(shape, device=device)

        for t in reversed(range(self.T)):
            t_batch = torch.full((shape[0],), t, device=device)
            noise_pred = self.forward(x, t_batch.float())

            alpha = self.alphas[t]
            alpha_cumprod = self.alphas_cumprod[t]
            beta = self.betas[t]

            mean = (1 / torch.sqrt(alpha)) * (
                x - (beta / torch.sqrt(1 - alpha_cumprod)) * noise_
            )

            if t > 0:
                noise = torch.randn_like(x)
                x = mean + torch.sqrt(beta) * noise
            else:
                x = mean

        return x
```

---

4. Flow Matching 구현

```python
class FlowMatching(nn.Module):
    """Conditional Flow Matching"""

    def __init__(self, input_dim, hidden_dim=256, cond_dim=512):
        super().__init__()

        self.time_embed = nn.Sequential(
            SinusoidalPosEmb(hidden_dim),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU()
        )

        self.cond_proj = nn.Linear(cond_dim, hidden_dim)

        self.net = nn.Sequential(
            nn.Linear(input_dim + hidden_dim * 2, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.SiLU(),
            nn.Linear(hidden_dim, input_dim)
        )

    def forward(self, x, t, cond):
        """속도 벡터 예측"""
        t_emb = self.time_embed(t)
        c_emb = self.cond_proj(cond)
        return self.net(torch.cat([x, t_emb, c_emb], dim=-1))

    def loss(self, x1, cond):
        """Flow Matching Loss"""
        B = x1.shape[0]

        # t ~ U[0, 1]
        t = torch.rand(B, device=x1.device)

        # x0 ~ N(0, I)
        x0 = torch.randn_like(x1)

        # 선형 보간: xt = (1-t)*x0 + t*x1
        t_expand = t[:, None]
        x_t = (1 - t_expand) * x0 + t_expand * x1

        # 실제 속도: v = x1 - x0
        v_target = x1 - x0

        # 예측 속도
        v_pred = self.forward(x_t, t, cond)

        return F.mse_loss(v_pred, v_target)
```

```python
    @torch.no_grad()
    def sample(self, cond, steps=10):
        """ODE Sampling (Euler)"""
        B = cond.shape[0]
        x = torch.randn(B, self.net[0].in_features - 512, device=co

        dt = 1.0 / steps
        for i in range(steps):
            t = torch.full((B,), i * dt, device=x.device)
            v = self.forward(x, t, cond)
            x = x + dt * v

        return x
```

---

5. VLA 모델 구현

```python
class SimpleVLA(nn.Module):
    """간소화된 VLA 모델 (RT1 스타일)"""

    def __init__(self, num_actions=7, num_bins=256, hidden_dim=512)
        super().__init__()

        # Vision: 사전학습 ViT
        from torchvision.models import vit_b_16, ViT_B_16_Weights
        vit = vit_b_16(weights=ViT_B_16_Weights.DEFAULT)
        self.vision_encoder = nn.Sequential(*list(vit.children())[:
        self.vision_proj = nn.Linear(768, hidden_dim)

        # Language: 간단한 임베딩
        self.text_encoder = nn.Sequential(
            nn.Embedding(30000, hidden_dim),
            nn.TransformerEncoder(
                nn.TransformerEncoderLayer(hidden_dim, 8, hidden_di
                num_layers=2
            )
        )

        # Fusion
        self.fusion = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(hidden_dim, 8, hidden_dim*4,
            num_layers=4
        )

        # Action heads
        self.action_heads = nn.ModuleList([
            nn.Linear(hidden_dim, num_bins) for _ in range(num_acti
        ])

    def forward(self, images, text_tokens):
        # Vision encoding
        B = images.shape[0]
        vis_feat = self.vision_encoder(images)  # (B, 197, 768)
        vis_feat = self.vision_proj(vis_feat)   # (B, 197, hidden_d

        # Text encoding
        txt_feat = self.text_encoder(text_tokens)  # (B, L, hidden_

        # Fusion
        combined = torch.cat([vis_feat, txt_feat], dim=1)
        fused = self.fusion(combined)

        # Pool and predict actions
        pooled = fused.mean(dim=1)  # (B, hidden_dim)

        action_logits = [head(pooled) for head in self.action_heads
        return action_logits

    def loss(self, images, text_tokens, action_labels):
```

```
    logits = self.forward(images, text_tokens)
    total_loss = 0
    for i, (logit, label) in enumerate(zip(logits, action_labels
        total_loss += F.cross_entropy(logit, label)
    return total_loss / len(logits)
```

---

6. 학습 루프 템플릿

```python
from torch.utils.data import DataLoader
from tqdm import tqdm
import wandb

def train_model(model, train_loader, val_loader, epochs=100, lr=1e-4
    """범용 학습 루프"""

    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer

    # Wandb 초기화
    wandb.init(project="robot-foundation", config={"lr": lr, "epochs

    best_val_loss = float('inf')

    for epoch in range(epochs):
        # Training
        model.train()
        train_loss = 0
        for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}/{epo
            optimizer.zero_grad()
            loss = model.compute_loss(batch)
            loss.backward()

            # Gradient clipping
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

            optimizer.step()
            train_loss += loss.item()

        train_loss /= len(train_loader)

        # Validation
        model.eval()
        val_loss = 0
        with torch.no_grad():
            for batch in val_loader:
                loss = model.compute_loss(batch)
                val_loss += loss.item()
        val_loss /= len(val_loader)

        # Logging
        wandb.log({
            "train_loss": train_loss,
            "val_loss": val_loss,
            "lr": scheduler.get_last_lr()[0]
        })

        # Checkpoint
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            torch.save(model.state_dict(), "best_model.pt")
```

```
        scheduler.step()
        print(f"Epoch {epoch+1}: Train Loss={train_loss:.4f}, Val L

    wandb.finish()
    return model
```

---

7. 하이퍼파라미터 권장값

7.1 VAE

파라미터	값	설명
β (KL 가중치)	0.0001 ~ 0.01	오디오는 낮게 시작
Learning Rate	1e-4	AdamW
Batch Size	16~64	GPU 메모리에 따라
Latent Dim	32~128	압축률과 품질 트레이드오프

7.2 Diffusion

파라미터	값	설명
T (타임스텝)	1000	표준값
β_min	1e-4	노이즈 스케줄 시작
β_max	0.02	노이즈 스케줄 끝
EMA Decay	0.9999	안정적 샘플링
CFG Scale	3~7	조건 준수도

7.3 VLA 모델

파라미터	값	설명
Learning Rate	1e-5 ~ 3e-4	사전학습 모델은 낮게
Epochs	20~30	BC는 많은 epoch 필요
LoRA Rank	16~64	메모리/성능 트레이드오프
Action Bins	256	표준값

---

8. 디버깅 팁

8.1 Loss가 줄어들지 않을 때

```python
# 1. 학습률 확인
for param_group in optimizer.param_groups:
    print(f"LR: {param_group['lr']}")

# 2. 그래디언트 확인
for name, param in model.named_parameters():
    if param.grad is not None:
        print(f"{name}: grad_norm={param.grad.norm():.4f}")

# 3. 데이터 확인
sample = next(iter(train_loader))
print(f"Input shape: {sample['image'].shape}")
print(f"Input range: [{sample['image'].min():.2f}, {sample['image']
```

8.2 메모리 부족

```python
# 1. Gradient checkpointing
model.gradient_checkpointing_enable()

# 2. Mixed precision
from torch.cuda.amp import autocast, GradScaler
scaler = GradScaler()

with autocast():
    loss = model(batch)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()

# 3. Batch size 줄이기 + Gradient accumulation
accumulation_steps = 4
for i, batch in enumerate(loader):
    loss = model(batch) / accumulation_steps
    loss.backward()
    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

---

9. 참고 자료

공식 코드 저장소

- **OpenVLA**: https://github.com/openvla/openvla

- **π0 (openpi)**: https://github.com/Physical-Intelligence/openpi
- **Octo**: https://github.com/octo-models/octo
- **DiffRhythm**: https://github.com/DiffRhythm/DiffRhythm

데이터셋

- **Open X-Embodiment**: https://robotics-transformer-x.github.io/
- **DROID**: https://droid-dataset.github.io/
- **Bridge V2**: https://rail-berkeley.github.io/bridgedata/

학습 자료

- Stanford CS 336: Robot Learning
- Berkeley EECS 106B: Robotic Manipulation
- CMU 16-831: Introduction to Robot Learning

본 강의 시리즈는 2025년 2월 기준 최신 연구를 반영하여 작성되었습니다.